

Ruminations on modular macros

Abstract

This paper explores why some audiences want macro support for modules, and also why some audiences don't. There's reason to believe the opposing sides of that argument fail to communicate their rationale to the other side; this paper boldly attempts to bridge some of those communication gaps.

Why is there such resistance for macro support in modules?

We should all know the downsides of macros by now; the probably worst thing about them is that they don't obey scopes. This leads to surprising effects when a library `#defines` a macro that doesn't even have a macro-like name. Examples of the common worst-offenders are

- X headers, which bring in macros 'Window' and 'Status'.
- Certain Windows headers, which bring in a macro 'interface', and even worse, macros 'min' and 'max'.
- Qt headers, which bring in macros 'signals' and 'slots'.

All of these macros, which are just selected examples, can cause surprising amounts of head-scratching. It can be very cumbersome to avoid them, because with the header inclusion model, they can come in very indirectly, via intermediate headers, and it's not straightforward nor obvious where and when to `#undef` them.

There have been multiple proposals throughout the years for avoiding macros polluting library client code. For the last couple of years, we have dismissed such isolated ideas with the response "modules will take care of that".

So, the design goal is that if there's a modern modular library, and a library client that doesn't use any legacy facilities, and doesn't `#include` or `#define` anything, but rather just imports modules, the program should in such cases avoid the macro problems completely. For some audiences, such "code hygiene" is of significant importance.

Why is there a desire to have macro support in modules?

The proponents for macro support in modules have provided the rationale that macro support is essential for libraries that want to export macros as part of their interface, and for gradual migration from headers to modules. For both of these rationales, it's assumed realistically that some libraries will export macros for a long foreseeable future; this is particularly the case for libraries that wish to allow clients to consume them both as modules and as headers.

Some such libraries are not all that easy to modify so as to stop them exporting macros. They can in some cases be widely-used system libraries that no company or programming team wishes to fork or otherwise modify, and due to their being widely-used, they need to support legacy users for a long time. If there's no macro support in modules, the benefits of modules can't be easily reaped for such libraries.

How can macros be provided with the Modules TS?

The suggestion how to 'export' macros from a library with the Modules TS seems to be to provide an additional header that can then be included, or alternatively, change an existing header to define the macros that the current header does and also import the right module. What is unclear to the author of this paper is

- What kind of deployment experience do we have with such an approach?
- Have the users who have deployed macro-supporting modules tried such an approach?
- How does that work with libraries that aren't easy to modify, like the aforementioned system headers? Qt? Windows? X11?
- What is the impact on build times?
- What is the impact on 'code hygiene'?

Are there compromises we can make to provide both 'code hygiene' and benefits of modules when having macros?

Allowing the user to choose whether the user wants macros

The proposals, or rather suggestions, for macro support in modules don't seem to provide a way to express "no, I don't want macros. Even if your library provides them, I don't want them." An argument can be made that the library author knows what she wants to provide, and that her API will not work if there's some way to be selective. It could equally well be said that the user knows best, and what that the user choice not to have macros is equally credibly the right choice for that user.

It wouldn't seem far-fetched to make a decision what happens when the library's choice and the user's choice are at odds. As an unbaked idea, a "no-macros" import that imports a module that wishes to export macros could be ill-formed. This is of course assuming that there is a "no-macros" import and a "macros are ok" import, separately.

It's perfectly plausible that there are audiences who think that a "macros are okay" import should not exist to begin with, because macros are, well, horrible, and should not be used. I'm not sure whether it's so plausible that there are really that many audiences who would think that we shouldn't have a "no macros" import, except for people who want a minimal set of features and think that macros are here to stay and are a necessary evil.

Allowing the module author to choose whether the author wants to export macros

We could certainly add mechanisms for exporting macros. The suggestions that I've seen written up are to provide

- a mechanism for `#defining` and exporting a macro.
- a mechanism for exporting all macros seen in the translation unit.

I must wonder how the authors of those suggestions came to the conclusion that that's the right set of mechanisms to have. I'm curious how the second doesn't lead to "transitive exports" and all the problems therein. I'm curious how the first handles being "purely additive". I'm a bit surprised that I haven't seen a proposal for exporting individual macros that have been `#defined` in headers that a module `#includes`. I'm curious how this works for macros that may or may not have been defined depending on platform/implementation detection, and whether there should be a conditional export, or whether I should consider preprocessor-programming an export depending on whether a macro has been defined.

When should we make changes?

This is perhaps the most interesting question of them all. Why should we worry about adding macro support in the Modules TS before we publish it? Why can't we do so in a subsequent revision?

We have deployment reports from Microsoft and Google, and that's great. I don't know whether I'm the only user who says this, but sadly

- those reports are next to useless to me.
- ..because they tell me nothing about how Modules work for my uses, which are guaranteed to be different from Microsoft's and Google's.

We have parties stating fairly adamantly that unless there's higher-class macro support in the Modules TS, it doesn't solve the problems of certain users. We have parties stating fairly adamantly that such higher-class macro support is not at all necessary, and is harmful. I can neither verify or falsify any such claims, until I can run two implementations implementing the same language extension on two different platforms. It seems to some extent unlikely that I'll get those two implementations unless we publish a TS.